Chapter 17: UML Class Diagrams

ENG3091: lecture notes

Khiem Nguyen

Contents

1 What is a UML class diagram? 2

2 A note on conventions 2

3 Class notation 2
- 3.1 Class structure . . . 3
- 3.2 Visibility . . . 3
- 3.3 Parameter direction . . . 4
- 3.4 Perspectives . . . 4

4 Relationships between classes 5
- 4.1 Dependency (“uses a”) . . . 6
- 4.2 Association (“knows a”) . . . 6
- 4.3 Aggregation (“has a”) . . . 7
- 4.4 Composition (“part of”) . . . 7
- 4.5 Inheritance, or generalization (“is a”) . . . 7
- 4.6 Realization (“implements”) . . . 8
- 4.7 Abstract classes . . . 8
- 4.8 Summary of relationships . . . 9

5 Multiplicity 10
- 5.1 Multiplicity examples . . . 10

6 Larger examples 11
- 6.1 Flights, pilots and airports . . . 11
- 6.2 Departments, lecturers, students and courses . . . 12

7 From the diagram to C++ 13

8 Past paper exam: shapes 15

9 Keep learning 16

> **i** How we use these notes
>
> A UML class diagram rarely fits on a single slide, so this chapter has no slides: we work through these notes directly in the lecture. They come as an HTML page and a PDF.

1 What is a UML class diagram?

The **Unified Modeling Language (UML)** is a standard way of drawing software designs. A **class diagram** shows a program's classes, what each class contains, and how the classes are related: exactly the relationships from Chapters 14 and 16.

A good class diagram lets someone write the code without asking you what you meant. That's the point of having rules.

2 A note on conventions

You'll find UML class diagrams on many websites, in textbooks and in drawing tools. The figures in these notes come from three places:

- Visual Paradigm's UML class diagram tutorial;
- GeeksforGeeks' article on UML class diagrams;
- diagrams drawn with **Mermaid**, a text-based diagram tool built into Quarto (see the source file `chap_17.qmd`).

These sources don't all draw things the same way. For example:

- GeeksforGeeks distinguishes a **directed association** (solid line, open arrow) from a plain association, and labels a dependency «use»; Visual Paradigm doesn't.
- Some figures put the class with the diamond on the left, others on the right. Either way, the diamond sits at the **whole**.
- Types may come after the name (`length : int`, the UML standard) or before it (`int length`, as in C++). Mermaid accepts both.
- Visual Paradigm writes C++-style types such as `Class6*` for a pointer; others write only the class name.
- Abstract classes and methods may be shown in *italics*, with the stereotype «abstract», or (in Mermaid) with an asterisk: `draw()*`.
- Tools colour their boxes and arrowheads differently. Colours carry no meaning.

> **!** The rule in this course
>
> **Any convention is acceptable, in your coursework and in the exam, as long as it follows the same principles and creates no unnecessary confusion.** In practice:
>
> 1. Each class is a box: its name, then its attributes, then its operations.
> 2. Visibility uses `+` (public), `-` (private) and `#` (protected).
> 3. Relationships follow the usual line styles: solid or dashed lines; a diamond at the *whole*, filled for composition and hollow for aggregation; a hollow triangle pointing at the *parent* (or interface).
> 4. Multiplicities are written at the ends of a relationship.
> 5. **Be consistent** within one diagram, and if you use anything unusual, label it.

If someone else could build the right C++ classes from your diagram without asking you a question, your diagram is good.

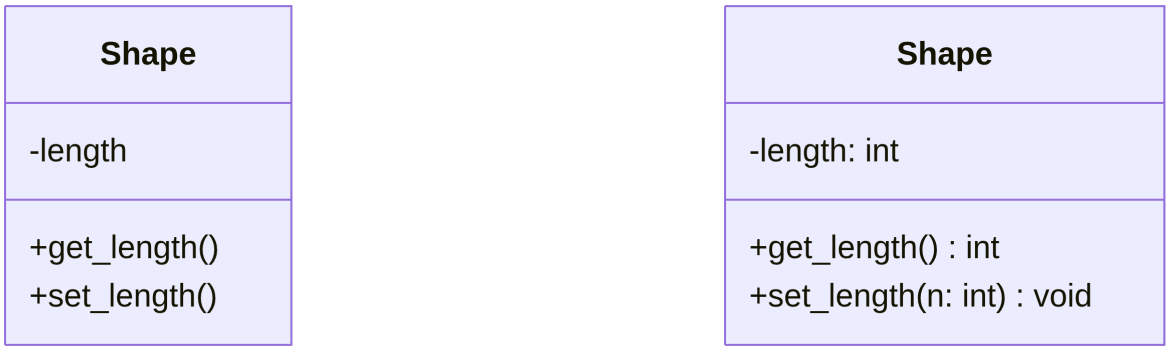
3 Class notation

A class represents a concept that encapsulates **state** (attributes) and **behaviour** (operations). Each attribute has a type, and each operation has a **signature**. Only the class **name** is mandatory.

3.1 Class structure

A UML class box has three compartments:

1. **the class name**, at the top;
2. **the attributes**, in the middle: these become **data members** in C++;
3. **the operations**, at the bottom: these become **member functions**.



Without types or signatures

With types and signatures

Figure 1: The same class, drawn with less and with more detail.

- **Attribute types** follow a colon: `-length : int`.
- **Return types** follow a colon at the end of the operation: `+get_length() : int`.
- **Parameter types** follow a colon after each parameter's name: `+set_length(n : int) : void`.

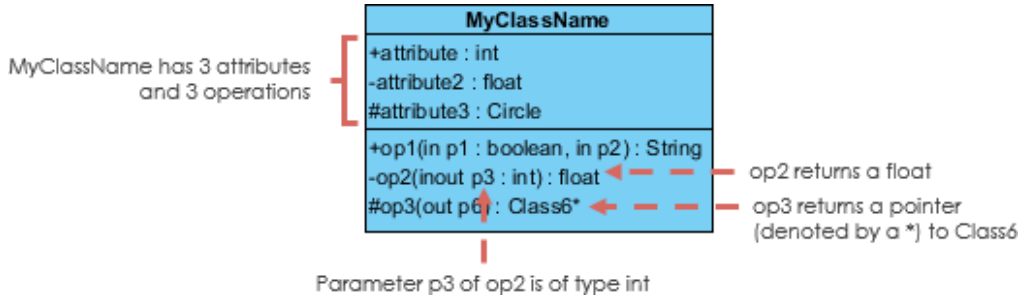


Figure 2: A class with typed attributes and operations. Source: Visual Paradigm.

	Without signatures	With full signatures
Attribute	`-length`	`-length : int`
Getter	`+get_length()`	`+get_length() : int`
Setter	`+set_length()`	`+set_length(n : int) : void`

3.2 Visibility

The symbol in front of an attribute or operation gives its **visibility**, which is its access level (Chapter 11):

- `+` means **public**;
- `-` means **private**;
- `#` means **protected** (Chapter 14).

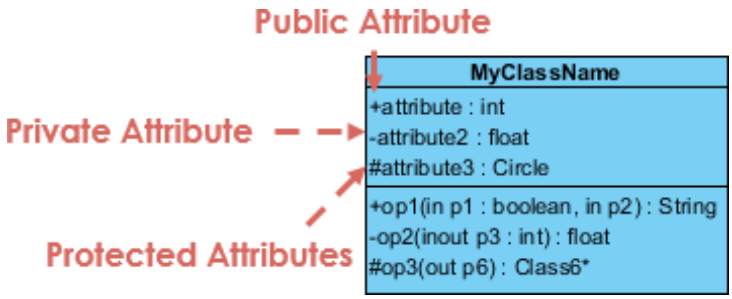


Figure 3: Attributes with different visibility. Source: Visual Paradigm.

3.3 Parameter direction

UML can mark each parameter as `in`, `out` or `inout`, written before the parameter's name. C++ has no such keywords; the parameter type says it instead:

UML	Direction	Meaning	In C++ (Chapter 07)
`in` (the default)	caller to operation	the operation only reads the value	pass by value, or by `const` reference: `void f(const Fraction& x)`
`out`	operation to caller	the operation fills in a value for the caller	pass by non-`const` reference: `void f(int& result)`
`inout`	both ways	the operation reads the value, then changes it	pass by non-`const` reference: `void f(int& x)`

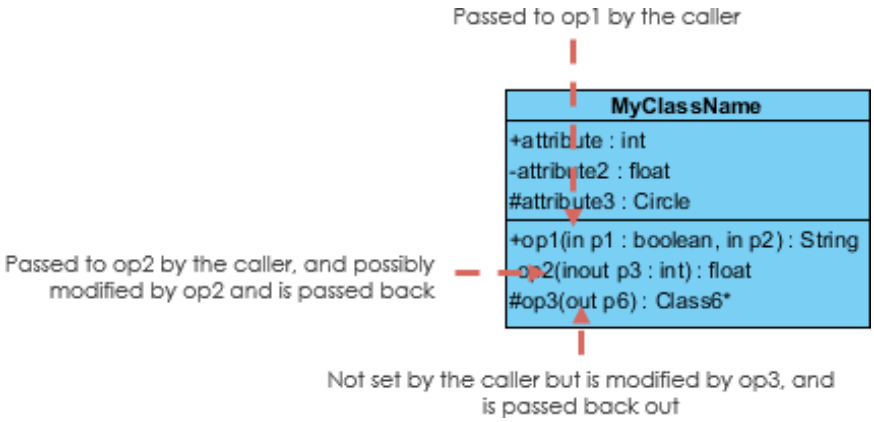


Figure 4: Parameter directions in a class. Source: Visual Paradigm.

3.4 Perspectives

The same diagram can be read at different levels of detail:

- **conceptual**: the concepts in the problem domain;
- **specification**: the interfaces of the classes (what they offer);
- **implementation**: how the classes implement those interfaces.

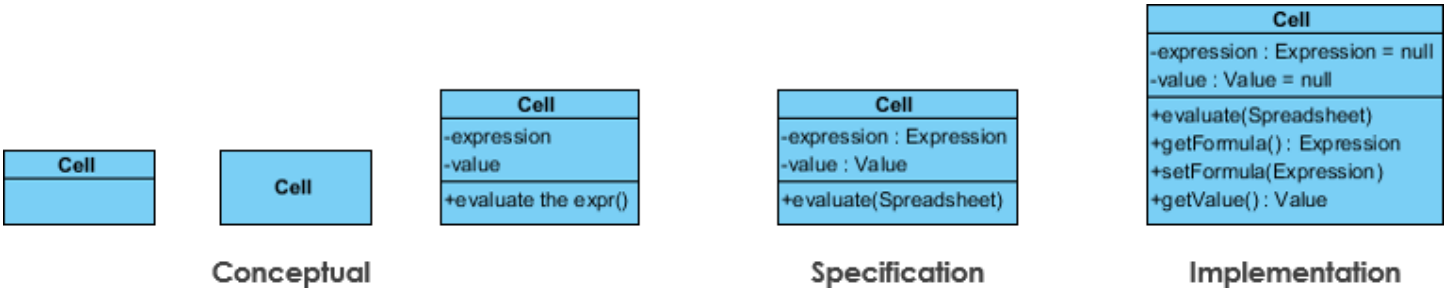
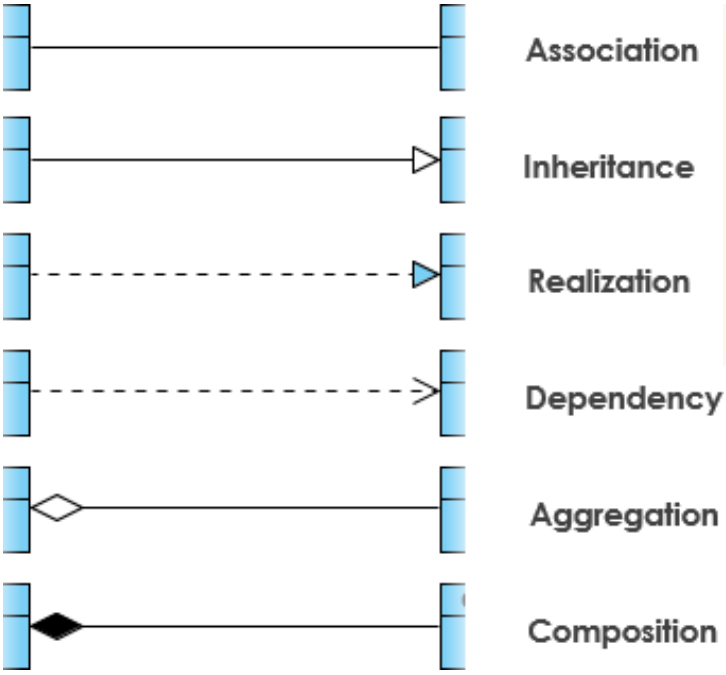


Figure 5: Perspectives of a class diagram. Source: Visual Paradigm.

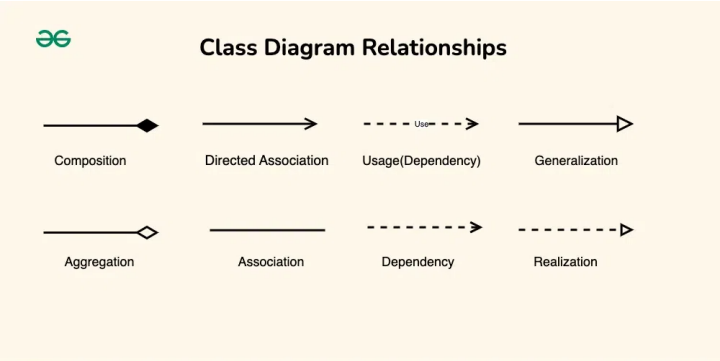
4 Relationships between classes

UML isn't just pretty pictures. Used correctly, a diagram says precisely how the code should be written, and code written from it reflects the designer's intent.

We've met all of these relationships already: inheritance (Chapter 14), realization of an interface (Chapter 15), and composition, aggregation, association and dependency (Chapter 16). Here's how two websites draw them. Notice the small differences, as discussed in Section 2:

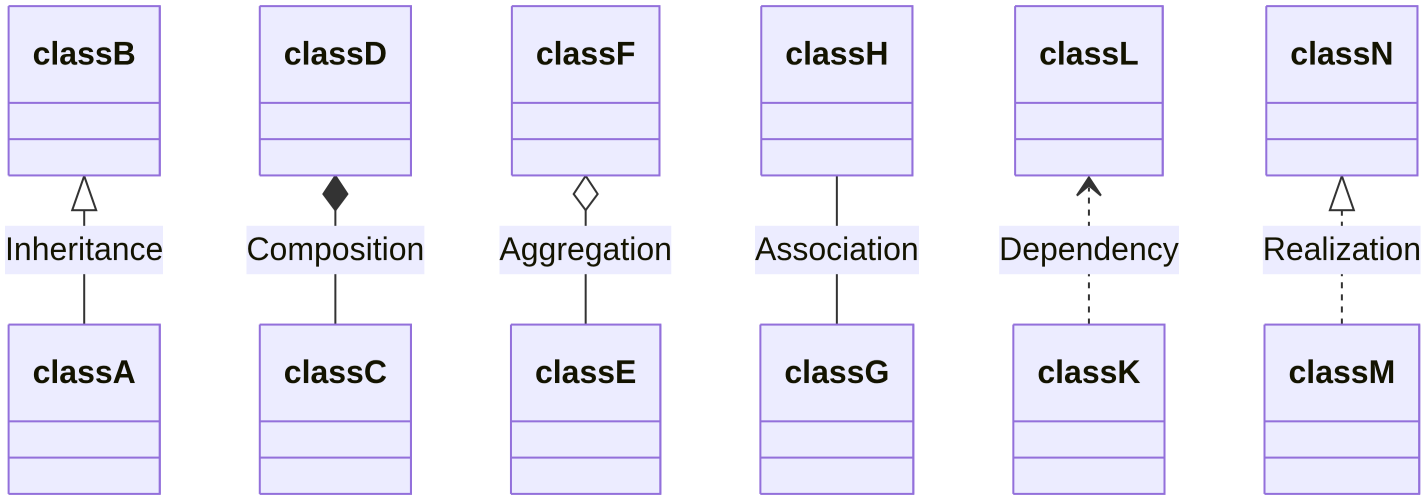


(a) Source: Visual Paradigm.



(a) Source: GeeksforGeeks.

And here they are in Mermaid:

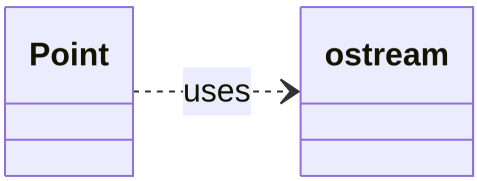


We'll go through them from the weakest to the strongest, using the classes from Chapter 16.

4.1 Dependency ("uses a")

A **dependency** is the weakest relationship: one class *uses* another for a moment, as a parameter or a local variable, without keeping it. If the class it uses changes, it may have to change too.

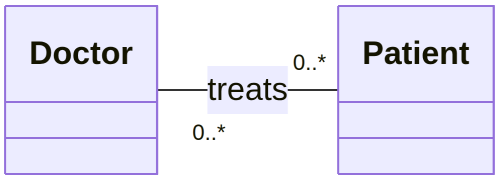
- **Logic:** "class A uses class B to do a task."
- **Notation:** a **dashed** line with an open arrow, pointing at the class being used.
- **Example (Chapter 16):** `Point` uses `std::ostream` in its `operator<<`. Likewise, a `Printer` uses a `Document` to print, but doesn't own it.



4.2 Association ("knows a")

An **association** is a structural link between two otherwise unrelated classes. It can be **unidirectional** (one class knows about the other) or **bidirectional** (both know about each other).

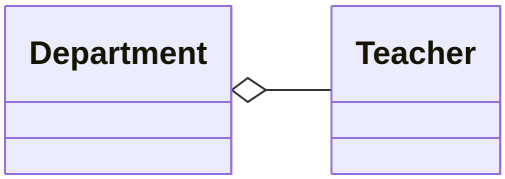
- **Logic:** "class A is associated with class B."
- **Notation:** a **solid** line. With an open arrowhead, it's a *directed* association: only the class at the tail knows about the other. With no arrowhead, it's bidirectional.
- **Example (Chapter 16):** a `Doctor` and a `Patient`. They know about each other, but neither owns the other.



4.3 Aggregation ("has a")

An **aggregation** is a whole–part relationship in which the parts can exist independently of the whole.

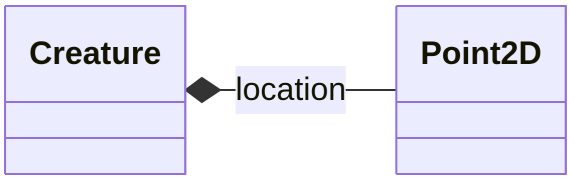
- **Logic:** "class A has a class B, but B can live without A."
- **Notation:** a solid line with a **hollow diamond** at the **whole**.
- **Example (Chapter 16):** a `Department` and its `Teacher`s. Closing the department doesn't destroy the teachers. Likewise a `Library` and its `Book`s.



4.4 Composition ("part of")

A **composition** is a strong whole–part relationship: the whole manages the part's lifetime. Destroy the whole, and the parts go with it.

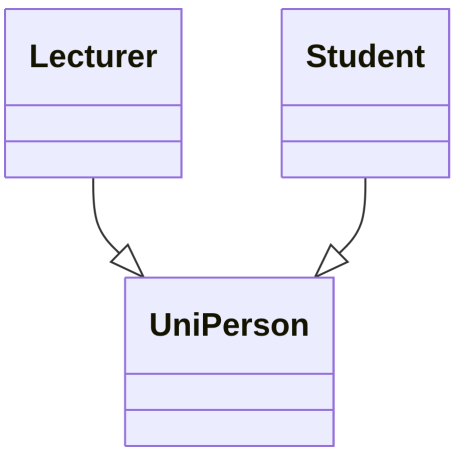
- **Logic:** "class B is an integral part of class A."
- **Notation:** a solid line with a **filled diamond** at the **whole**.
- **Example (Chapter 16):** a `Creature` and its `Point2D` location. Likewise a `House` and its `Room`s: demolish the house, and the rooms are gone.



4.5 Inheritance, or generalization ("is a")

Inheritance means a derived (child) class inherits the attributes and operations of its base (parent) class: it's a specialized version of a more general concept.

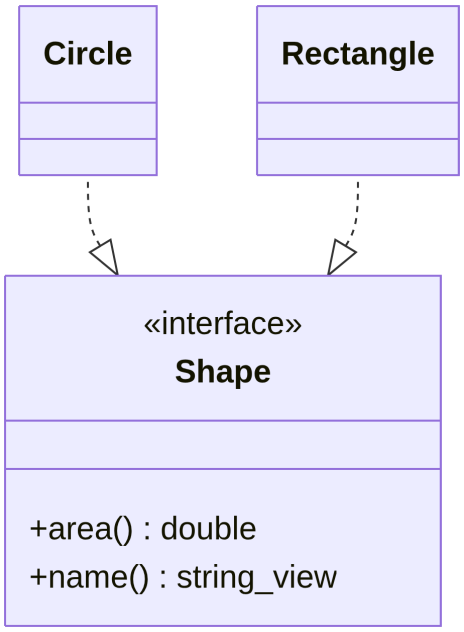
- **Logic:** "class B is a kind of class A."
- **Notation:** a solid line with a **hollow triangle** pointing at the **parent**.
- **Example (Chapter 14):** a `Lecturer` is a `UniPerson`, and so is a `Student`.



4.6 Realization ("implements")

Realization connects an **interface** to a class that implements it. The class *realizes* the contract that the interface defines. In C++, an interface is an abstract class with only pure virtual functions (Chapter 15).

- **Logic:** "class A implements everything interface B promises."
- **Notation:** a **dashed** line with a **hollow triangle** pointing at the interface.
- **Example (Chapter 15):** `Circle` and `Rectangle` implement the `Shape` interface.

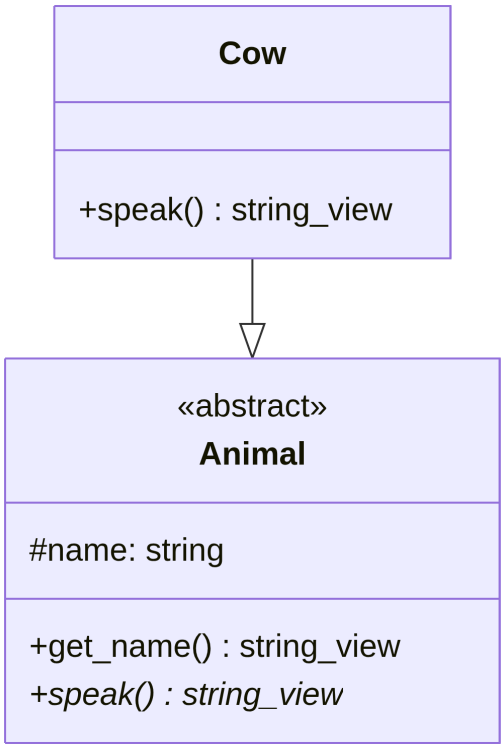


> **Realization or inheritance?**
>
> Strictly, realization is for **interfaces**. A class derived from an abstract class that *also* has data or implemented functions (like Chapter 15's `Animal`) is drawn with ordinary inheritance: a solid line. Many websites and tools blur this distinction. Following Section 2, either is accepted, as long as you're consistent.

4.7 Abstract classes

An abstract class can't be instantiated (Chapter 15), and the diagram should make that visible. There are two common ways:

- **italics**: the class name in *italics*, and each abstract (pure virtual) operation in *italics* too;
- **a stereotype**: the word «abstract» above the class name. It's clearer in hand-drawn diagrams, where italics are hard to see.



In Mermaid, `speak()*` marks an abstract operation.

> **Abstract classes in the paper exam**
>
> Handwriting has no italics, so use either of these:
>
> - write «abstract» (or `<<abstract>>`) above the class name;
> - surround the name with asterisks, `*ClassName*`, as if it were italic (Markdown style).

4.8 Summary of relationships

Relationship	In words	Lifetime	Notation
Dependency	“uses a” (for a moment)	independent	dashed line, open arrow (- - >)
Association	“knows a”	independent	solid line ( ), with an open arrow if directed
Aggregation	“has a”	independent	solid line, hollow diamond at the whole ( )
Composition	“part of”	the whole manages the part	solid line, filled diamond at the whole ( )
Inheritance	“is a”	the derived object contains the base part	solid line, hollow triangle at the parent ( )
Realization	“implements”	(a contract)	dashed line, hollow triangle at the interface (- -)

5 Multiplicity

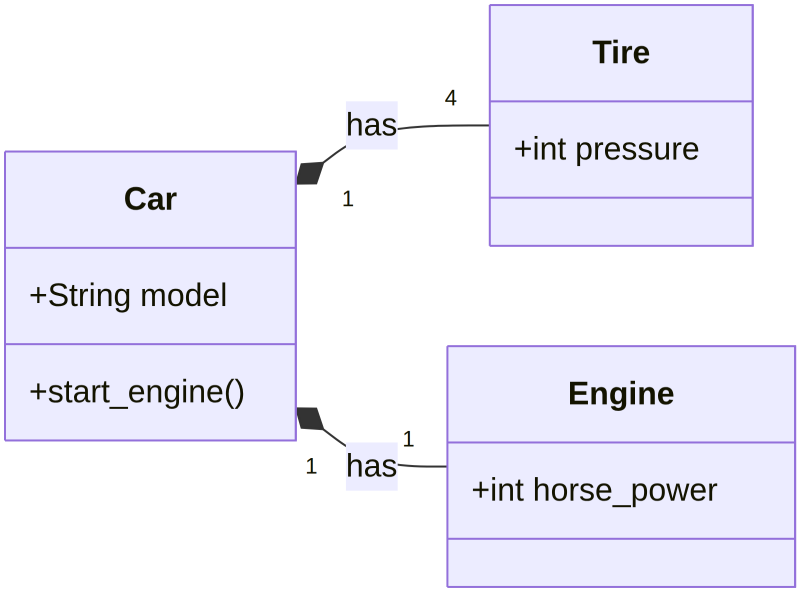
Multiplicity (often called **cardinality**) says how many objects of one class can be linked to one object of the other. It's written at the ends of a relationship.

Notation	Meaning	Example
`0..1`	zero or one	an `Employee` may have a `Desk`
`1`	exactly one	a `Car` has exactly one `Engine`
`0..*`	zero or more	a `Folder` contains any number of `Files`
`1..*`	one or more	a `Team` has at least one `Player`
`*`	many	short for `0..*`
`n`	exactly n	a `Tricycle` has exactly `3` `Wheels`
`m..n`	between m and n	a `Jury` has between `6` and `12` members

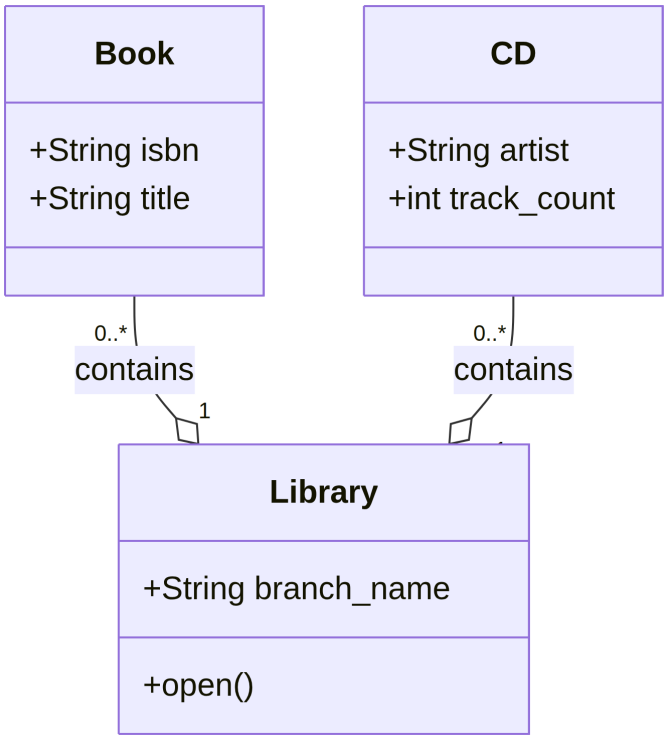
5.1 Multiplicity examples

As discussed in Section 2, it's fine to write an attribute's type before its name, as below, or after it.

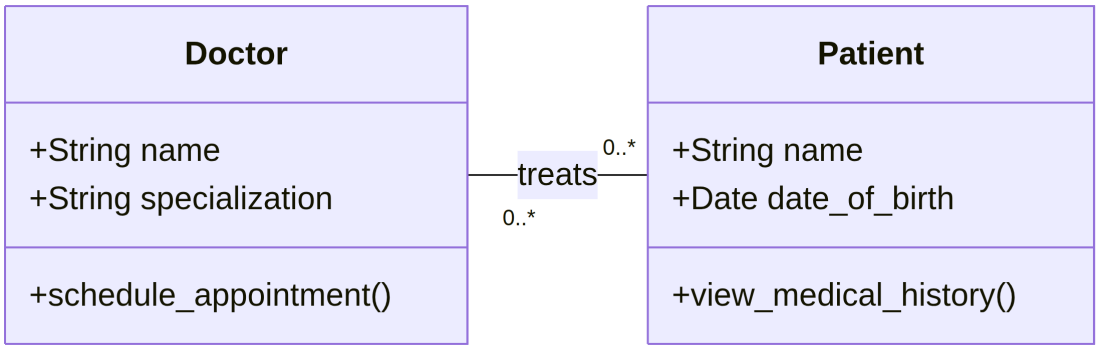
Example 1. A `Car` has four `Tires` and one `Engine`: compositions with fixed multiplicities.



Example 2. A `Library` holds any number of `Books` and `CDs`: aggregations with "many".



Example 3. A `Doctor` treats many `Patients`, and a `Patient` may see many `Doctors`: a many-to-many association.

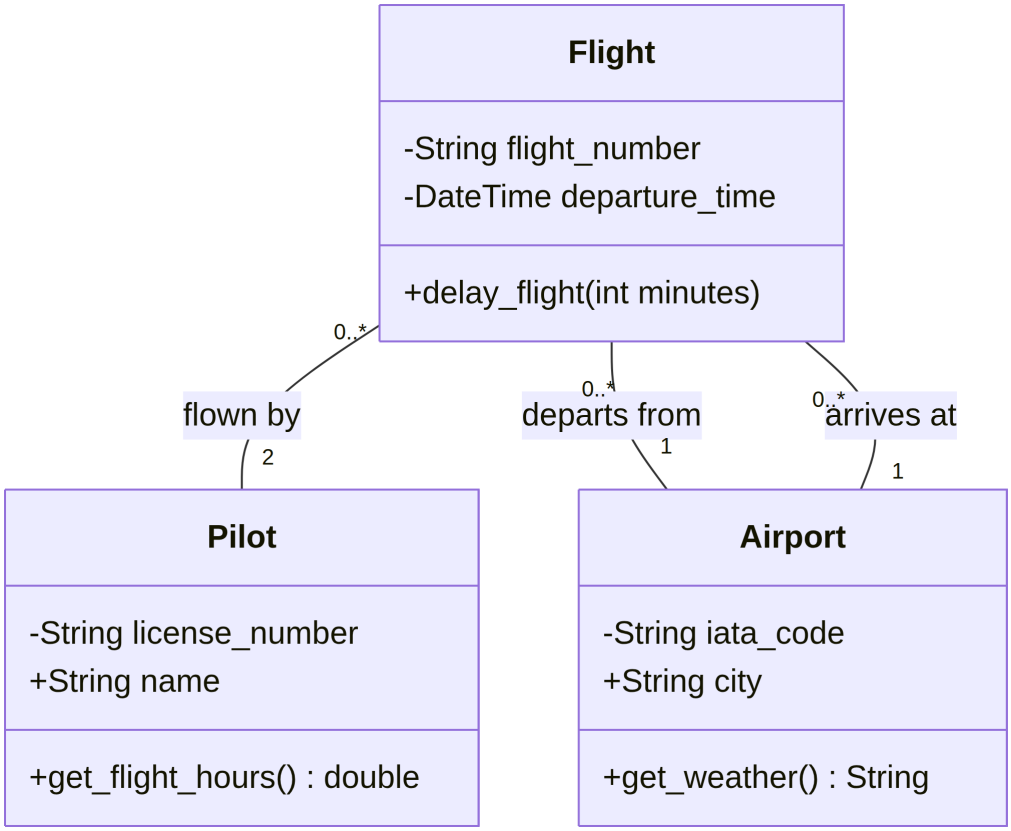


6 Larger examples

6.1 Flights, pilots and airports

`Flight`, `Pilot` and `Airport` are linked by **associations**:

- a `Pilot` *works* a `Flight`: a relationship of service, not a structural "part of";
- an `Airport` hosts many `Flights`, and each `Flight` departs from one `Airport` and arrives at another.

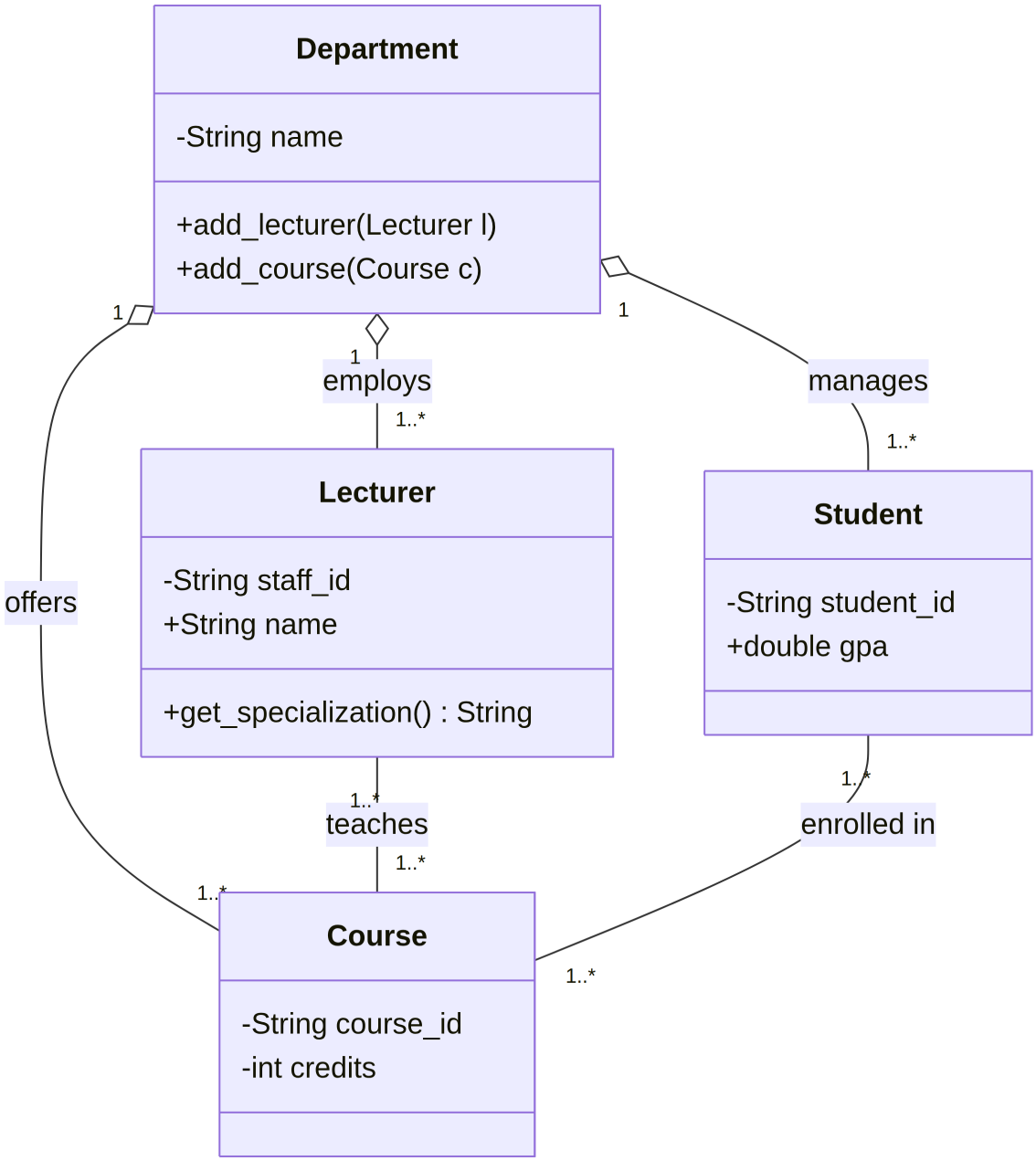


6.2 Departments, lecturers, students and courses

1. A `Department` has many `Lecturers`.
2. A `Department` has many `Students`.
3. A `Department` offers many `Courses`.
4. Several `Lecturers` can teach one `Course`.
5. One `Lecturer` can teach several `Courses`.
6. One `Student` can take many `Courses`.
7. One `Course` can have many `Students`.

Statements 1 to 3 describe "part of the department": aggregations, since lecturers, students and courses don't disappear if the department is reorganized. Statements 4 to 7 describe *providing* and *using* a service: associations.

> **Note**
>
> In everyday English we could say "has" for every one of these sentences. But a student rarely moves between departments during their studies, while lecturers change courses quite often. The lecturer–course and student–course links are interactions that come and go, which is what associations describe.



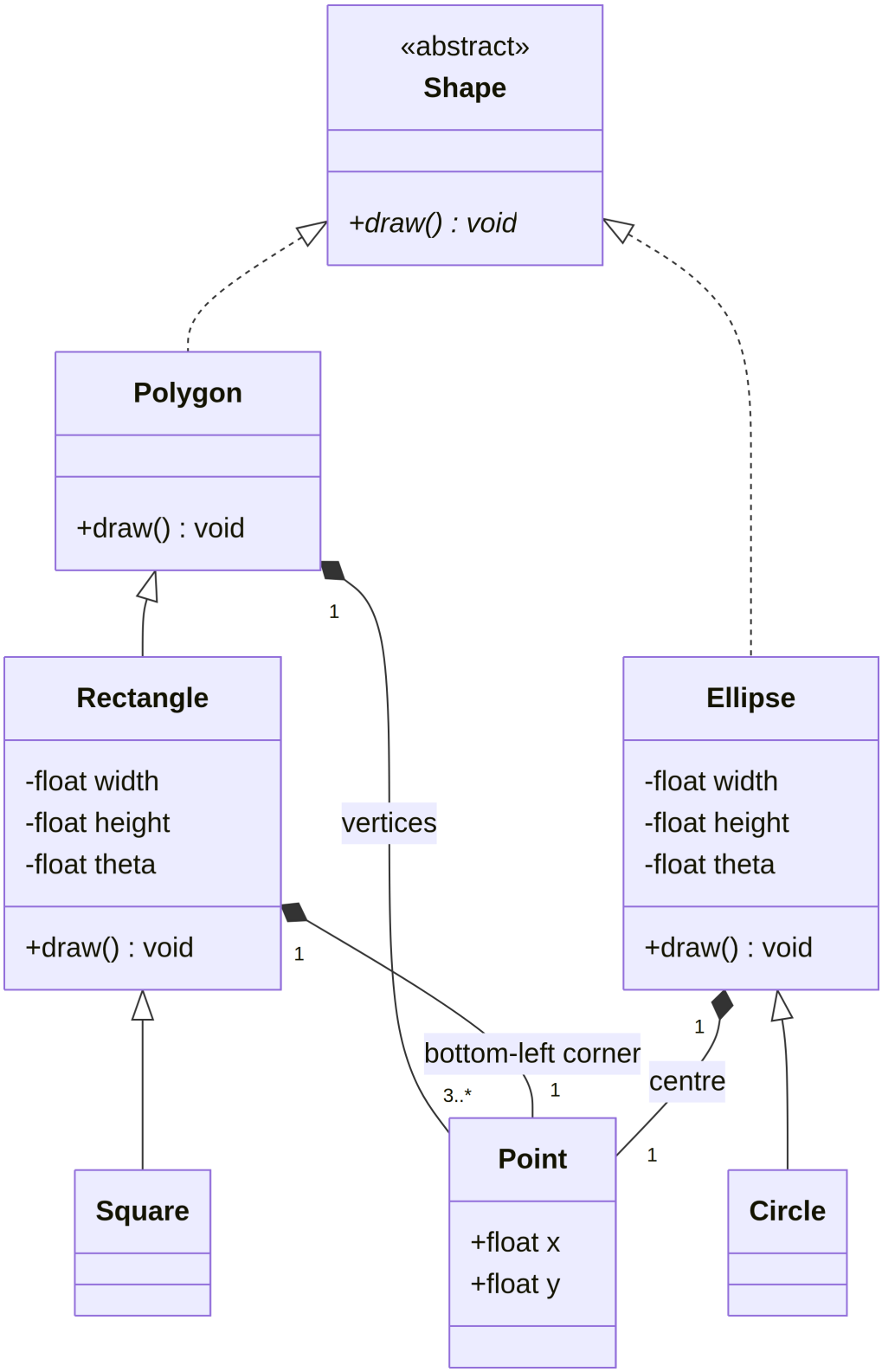
7 From the diagram to C++

This is what the exam asks of you: read a diagram, and write the classes, or the other way round. The usual translations:

In the diagram	In C++
class box	`class Name { ... };`
attribute `-x : double`	private data member `double x{};`
operation `+f(n : int) : void`	public member function `void f(int n);`
composition, multiplicity `1`	an ordinary data member: `Point2D location{};`
composition, multiplicity `*`	a `std::vector` of members, or `std::unique_ptr`s the class owns
aggregation	a reference or pointer to an outside object; for many, `std::vector<std::reference_wrapper<T>>`
association	pointer members (for many, `std::vector<const T*>`), on one or both sides

In the diagram	In C++
dependency	a function parameter or a local variable, not a member
inheritance	`class Derived : public Base`
realization	inherit from an interface and override every pure virtual function
abstract operation	a pure virtual function: `virtual double area() const = 0;`

8 Past paper exam: shapes



- `Polygon` and `Ellipse` must list `+draw() : void`: they implement the abstract operation of `Shape`.
- Repeating `+draw() : void` in `Square` and `Circle` is allowed, but unnecessary: they inherit it.
- `Shape` is abstract, with an abstract `draw()`. The dashed realization arrows from `Polygon` and `Ellipse` follow the "implements" reading of Section 2. Solid inheritance arrows would be accepted too.

9 Keep learning

1. learncpp.com, Chapter 23: object relationships, the C++ side of every relationship drawn here.
2. Visual Paradigm's UML class diagram tutorial.
3. GeeksforGeeks on UML class diagrams.
4. The Mermaid class diagram syntax, to draw your own diagrams from text.